Суми за допомогою рекурсії: 5.4.1–5.4.4

Звіт • Джамалов Олександр Олегович • PI-11

За допомогою рекурсивних функцій обчислити значення суми чи добутку.

В одній програмі слід вивести результати звертання до п’яти функцій, які реалізують такі способи:

1) обчислення здійснюються на рекурсивному спуску, значення параметра в рекурсивних викликах – спадає;
2) обчислення здійснюються на рекурсивному спуску, значення параметра в рекурсивних викликах – зростає;
3) обчислення здійснюються на рекурсивному підйомі, значення параметра в рекурсивних викликах – спадає;
4) обчислення здійснюються на рекурсивному підйомі, значення параметра в рекурсивних викликах – зростає;
5) обчислення здійснюється ітераційним способом (для контролю правильності рекурсивних способів).

Всі п’ять результатів мають збігатися. Всю необхідну функціям інформацію слід передавати лише за допомогою параметрів.

Варіант 7.

$$\sum_{k=1}^{N} \frac{1}{(2k+1)^2}$$

a(k) = 1/(2k+1)². Початкова сума дорівнює 0.
5.4.1: upDown(N); 5.4.2: upUp(1,N). Додавання після виклику.
5.4.3: downDown(N,0); 5.4.4: downUp(1,N,0). Додавання до виклику.
Для захисту стека в програмі прийнято 1 ≤ N ≤ 1000.
Кожний рекурсивний спосіб має N + 1 рівнів, включно з базовим.

Структура: main → 4 рекурсивні функції, iterative; усі → term

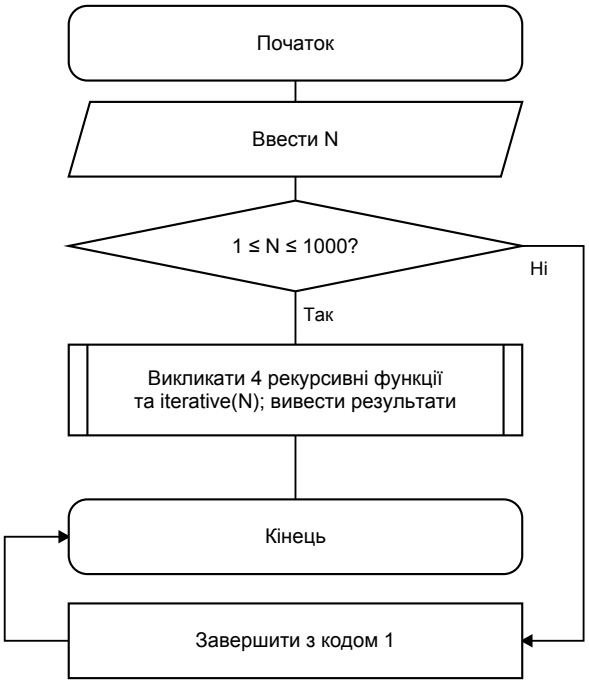
main	upDown, upUp, downDown, downUp, iterative → term

Git-репозиторій: _____

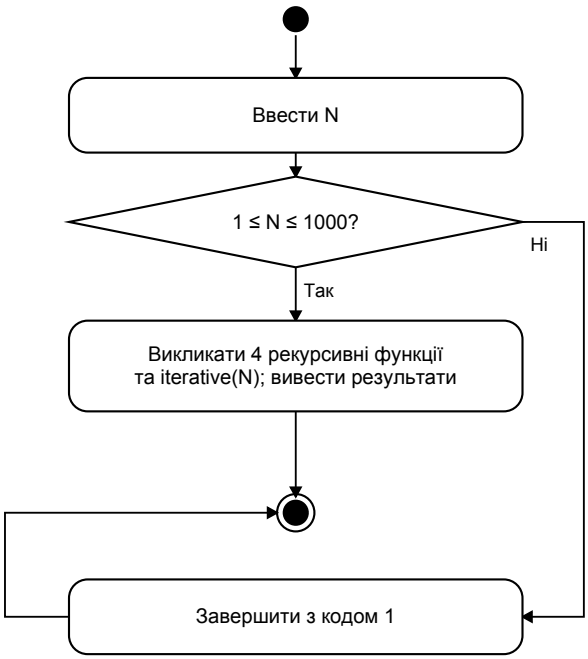
Основна програма та ітераційна перевірка

main()

Блок-схема

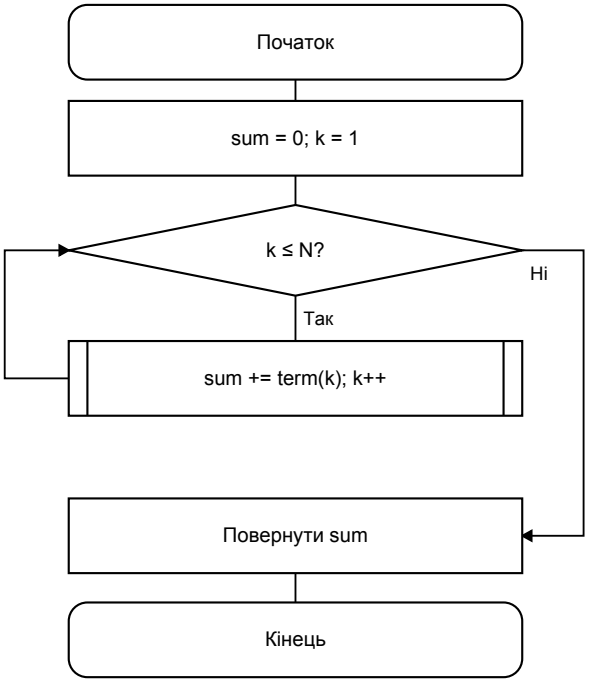


UML-діаграма діяльності

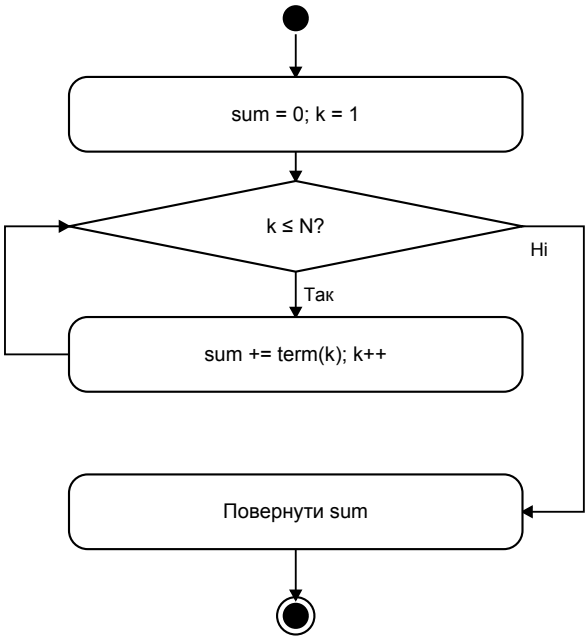


iterative(N)

Блок-схема



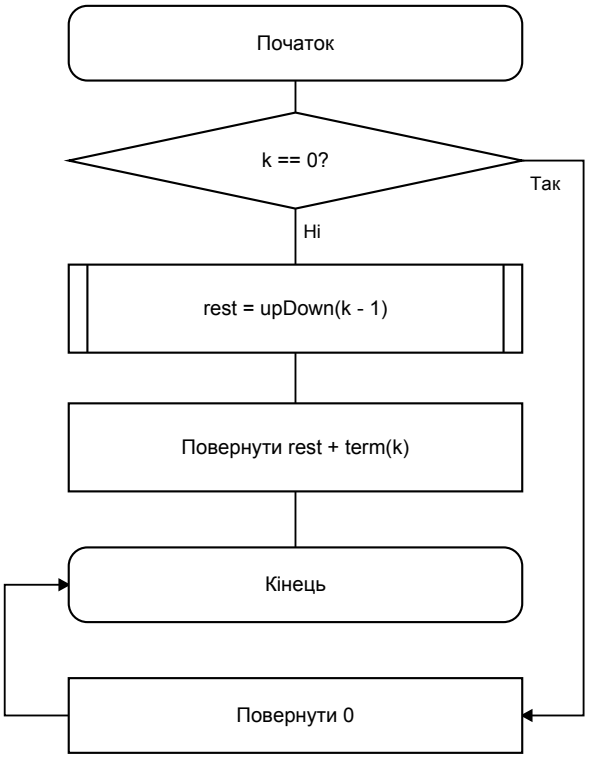
UML-діаграма діяльності



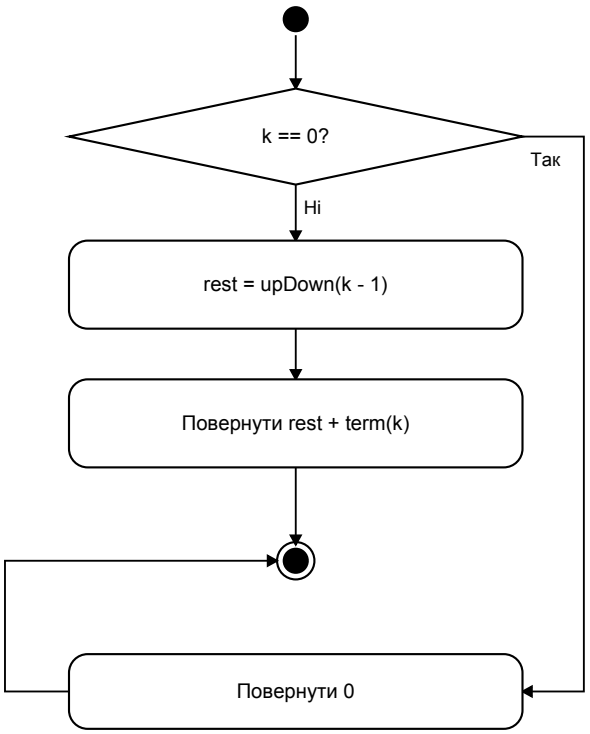
Рекурсивний підйом

5.4.1 • upDown(k): параметр спадає

Блок-схема

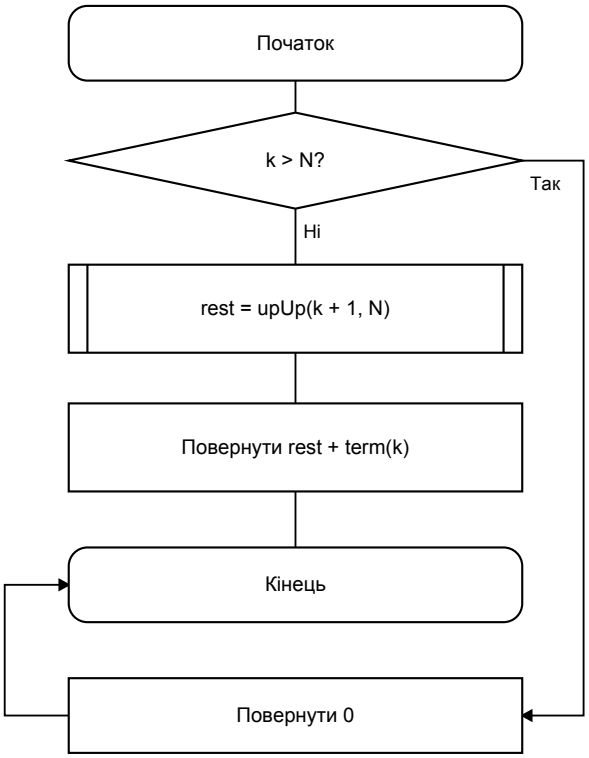


UML-діаграма діяльності

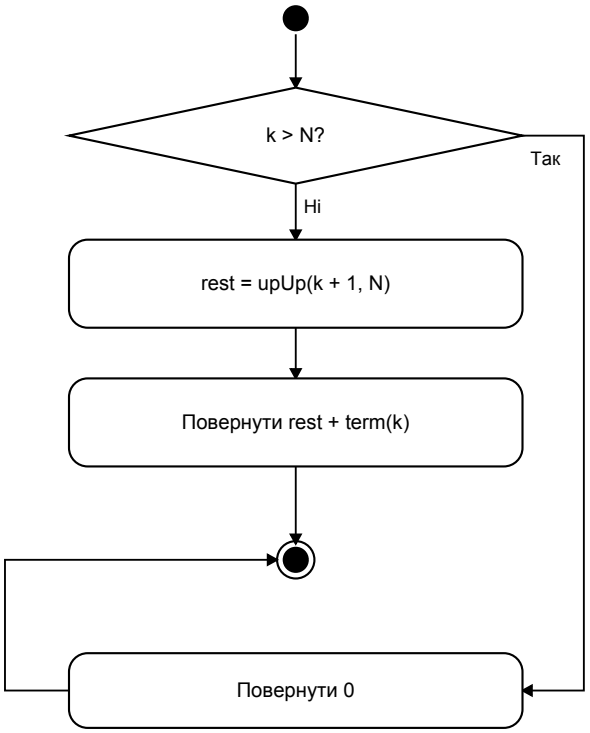


5.4.2 • upUp(k,N): параметр зростає

Блок-схема



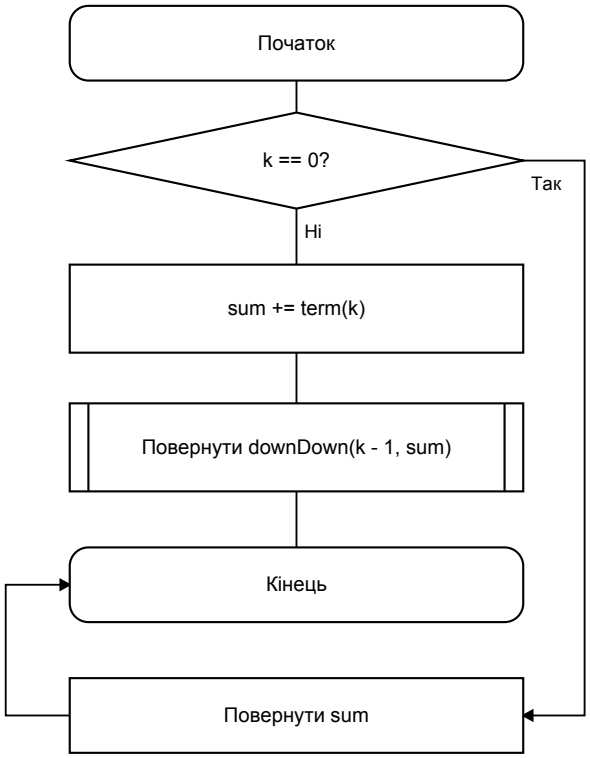
UML-діаграма діяльності



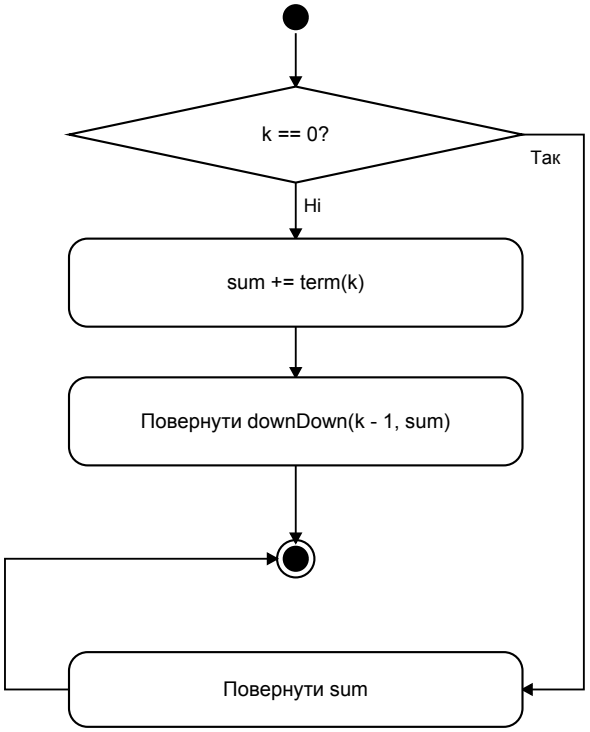
Рекурсивний спуск

5.4.3 • downDown(k,sum): параметр спадає

Блок-схема

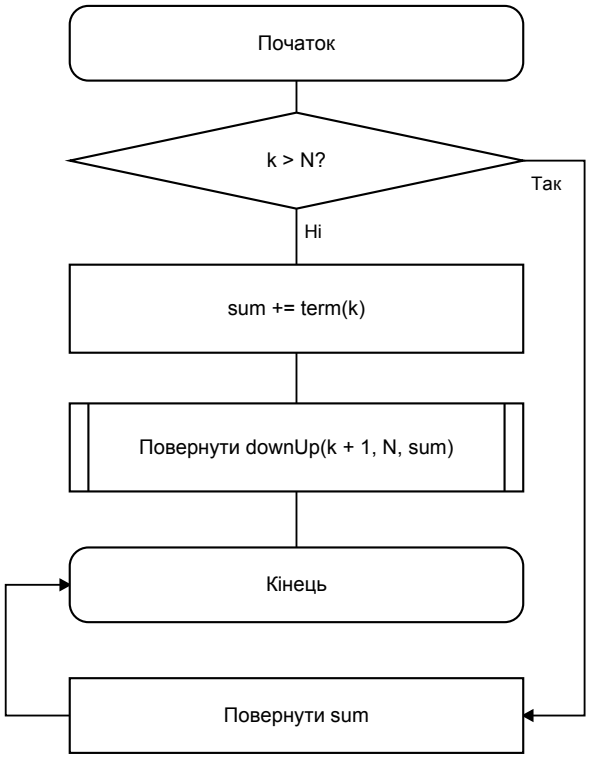


UML-діаграма діяльності

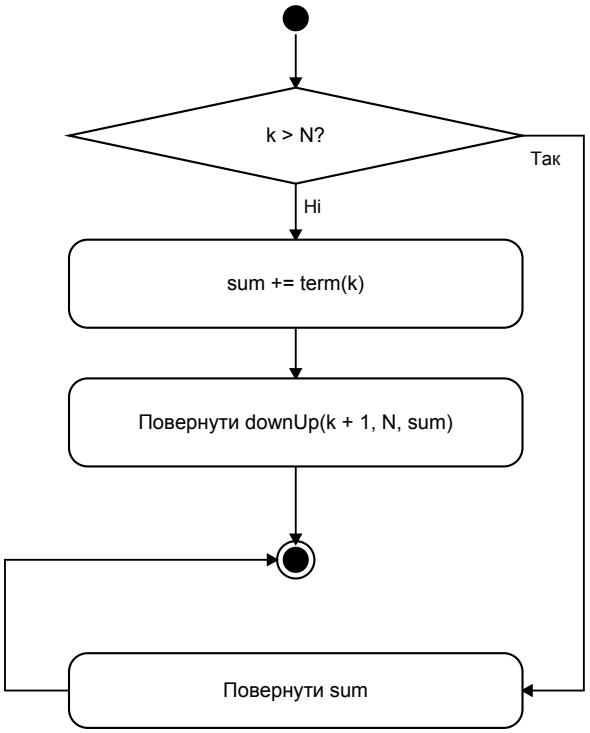


5.4.4 • downUp(k,N,sum): параметр зростає

Блок-схема



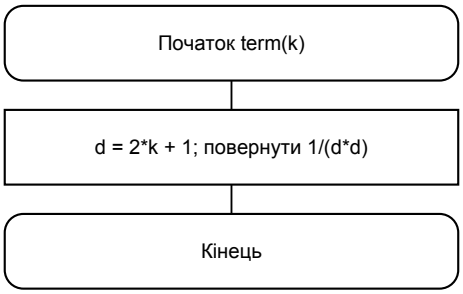
UML-діаграма діяльності



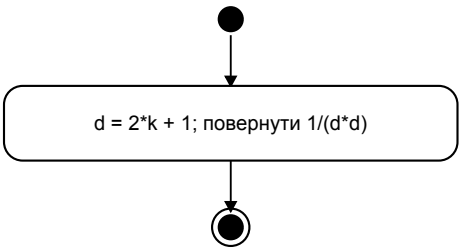
Функція доданка та результати

term(k)

Блок-схема



UML-діаграма діяльності



Фактичний вивід unit-тесту

```
PASS N=1 (all five methods)
PASS N=2 (all five methods)
PASS N=5 (all five methods)
PASS N=20 (all five methods)
PASS N=100 (all five methods)
PASS N=1000 (all five methods)
5.4: 6/6 passed
```

Приклад: N = 5

```
N (1..1000): 5.4.1 = 0.192129416199
5.4.2 = 0.192129416199
5.4.3 = 0.192129416199
5.4.4 = 0.192129416199
iter  = 0.192129416199
```

Усі 6 тестів порівнюють п’ять способів з еталоном long double.
Допуск 10^(-12) враховує різний порядок додавання.
Висновок: усі чотири рекурсивні способи дають правильну суму.

Текст програми та unit-тести

lab_5_4.cpp

```
#include <iomanip>
#include <iostream>
using namespace std;

double term(int k)
{
    const double d = 2.0 * k + 1;
    return 1 / (d * d);
}

// 5.4.1: recursive ascent, decreasing parameter.
double upDown(int k)
{
    if (k == 0)
        return 0;
    const double rest = upDown(k - 1);
    return rest + term(k);
}

// 5.4.2: recursive ascent, increasing parameter.
double upUp(int k, int n)
{
    if (k > n)
        return 0;
    const double rest = upUp(k + 1, n);
    return rest + term(k);
}

// 5.4.3: recursive descent, decreasing parameter.
double downDown(int k, double sum)
{
    if (k == 0)
        return sum;
    sum += term(k);
    return downDown(k - 1, sum);
}

// 5.4.4: recursive descent, increasing parameter.
double downUp(int k, int n, double sum)
{
    if (k > n)
        return sum;
    sum += term(k);
    return downUp(k + 1, n, sum);
}

// Iterative reference.
double iterative(int n)
{
    double sum = 0;
    for (int k = 1; k <= n; ++k)
        sum += term(k);
    return sum;
}

#ifndef UNIT_TEST
int main()
{
    int n;
```

Текст програми та unit-тести

```
    cout << "N (1..1000): ";
    if (!(cin >> n) || n < 1 || n > 1000)
        return 1;
    cout << fixed << setprecision(12)
         << "5.4.1 = " << upDown(n) << '\n'
         << "5.4.2 = " << upUp(1, n) << '\n'
         << "5.4.3 = " << downDown(n, 0) << '\n'
         << "5.4.4 = " << downUp(1, n, 0) << '\n'
         << "iter  = " << iterative(n) << '\n';
    return 0;
}
#endif
```

test_5_4.cpp

```
#define UNIT_TEST
#include "lab_5_4.cpp"
#include <cmath>

int main()
{
    int failed = 0;
    for (int n : {1, 2, 5, 20, 100, 1000})
    {
        long double expected = 0;
        for (int j = 1; j <= n; ++j)
            expected += 1.0L / ((2.0L * j + 1) * (2.0L * j + 1));
        const double results[] = {upDown(n), upUp(1, n),
            downDown(n, 0), downUp(1, n, 0), iterative(n)};
        bool ok = true;
        for (double value : results)
            ok = ok && std::abs(value - expected) < 1e-12;
        cout << (ok ? "PASS" : "FAIL") << " N=" << n
             << " (all five methods)\n";
        if (!ok) ++failed;
    }
    cout << "5.4: " << 6 - failed << "/6 passed\n";
    return failed ? 1 : 0;
}
```